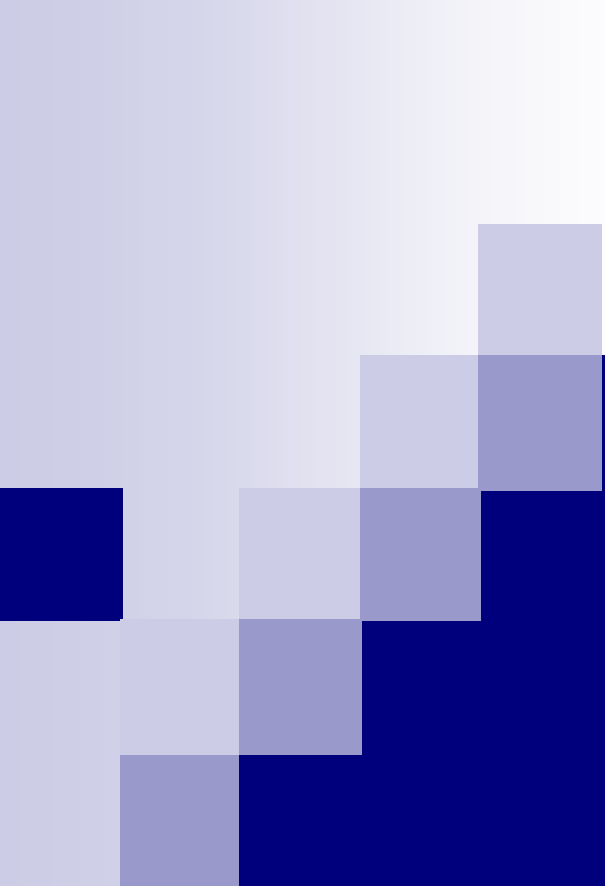




Software Requirements and SQA



Lecture # 7

Second Phase of the Course

- In this phase, we will discuss the importance, impact, practices, and techniques of software quality assurance as they relate to different activities in the software development processes
- We'll begin with requirements

Introduction

- Requirements form the basis for all software products
- Requirements engineering is the process, which enables us to systematically determine the requirements for a software product

Requirement

- Something required, something wanted or needed
 - Webster's dictionary
- Difference between *wanted* and *needed*

An Important Point

- We are discussing requirements with reference to quality assurance and management
- Every item we discuss, try to associate issues of quality assurance with it

Importance of Requirements

- The hardest single part of building a software system is deciding what to build...No other part of the work so cripples the resulting system if done wrong. No other part is difficult to rectify later
 - Fred Brooks

Software Requirements - 1

- A complete description of *what* the software system will do without describing *how* it will do it
- External behavior

Software Requirements - 2

- Software requirements may be:
 - Abstract statements of services and/or constraints
 - Detailed mathematical functions
 - Part of the bid of contract
 - The contract itself
 - Part of the technical document, which describes a product

IEEE Definition

- A condition or capability that must be met or possessed by a system...to satisfy a contract, standard, specification, or other formally imposed document
 - IEEE Std 729

Sources of Requirements

- Stakeholders
 - People affected in some way by the system
 - Stakeholders describe requirements at different levels of detail
- Documents
- Existing system
- Domain/business area

Examples of Requirements - 1

- The system shall maintain records of all payments made to employees on accounts of salaries, bonuses, travel/daily allowances, medical allowances, etc.
- The system shall interface with the central computer to send daily sales and inventory data from every retail store

Examples of Requirements - 2

- The system shall maintain records of all library materials including books, serials, newspapers and magazines, video and audio tapes, reports, collections of transparencies, CD-ROMs, DVDs, etc.

Kinds of Software Requirements

- Functional requirements
- Non-functional requirements
- Domain requirements
- Inverse requirements
- Design and implementation constraints

Functional Requirements - 1

- Statements describing what the system does
- Functionality of the system
- Statements of services the system should provide
 - Reaction to particular inputs
 - Behavior in particular situations

Functional Requirements - 2

- Sequencing and parallelism are also captured by functional requirements
- Abnormal behavior is also documented as functional requirements in the form of exception handling

Non-Functional Requirements - 1

- Most non-functional requirements relate to the system as a whole. They include constraints on timing, performance, reliability, security, maintainability, accuracy, the development process, standards, etc.

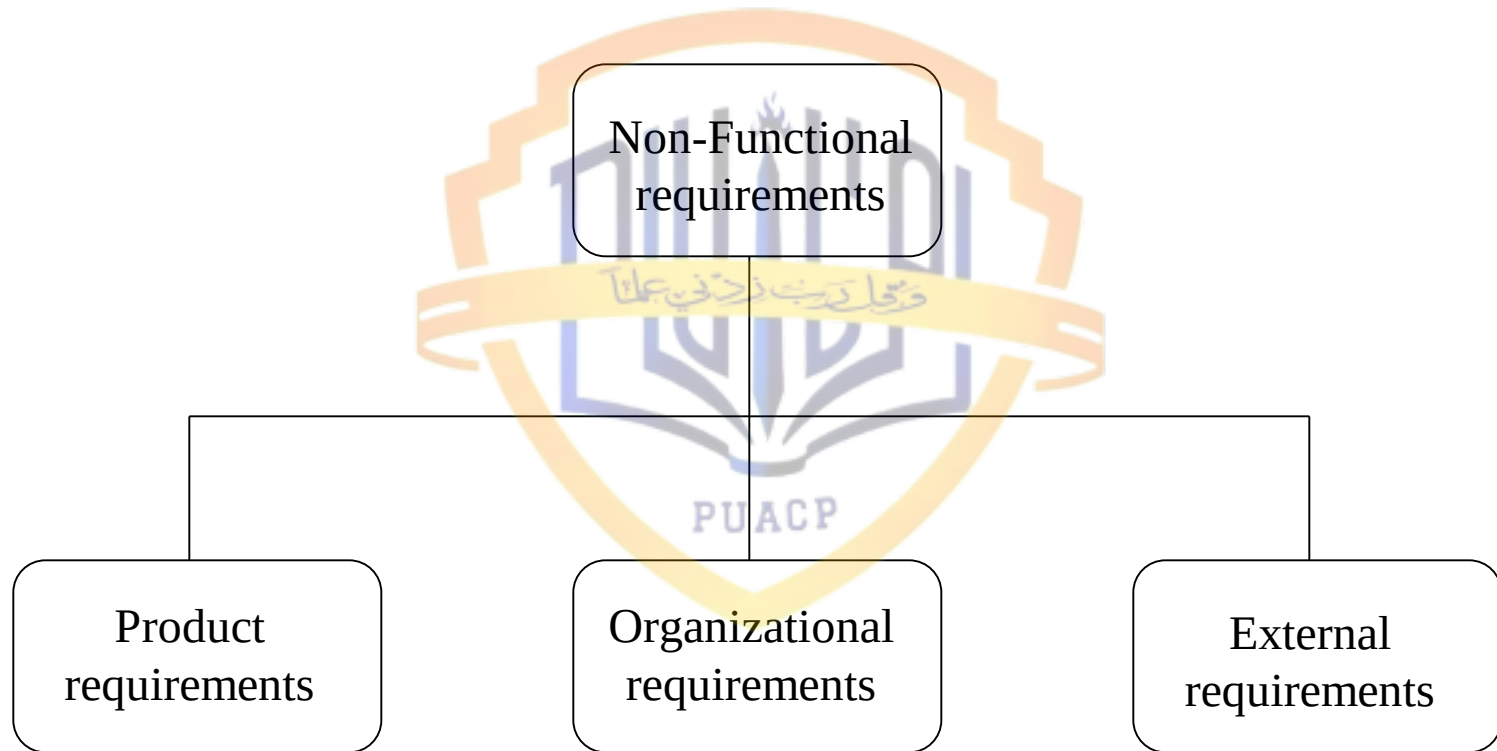
Non-Functional Requirements - 2

- They are often more critical than individual functional requirements
- Capture the emergent behavior of the system, that is they relate to system as a whole
- Many non-functional requirements describe the quality attributes of the software product

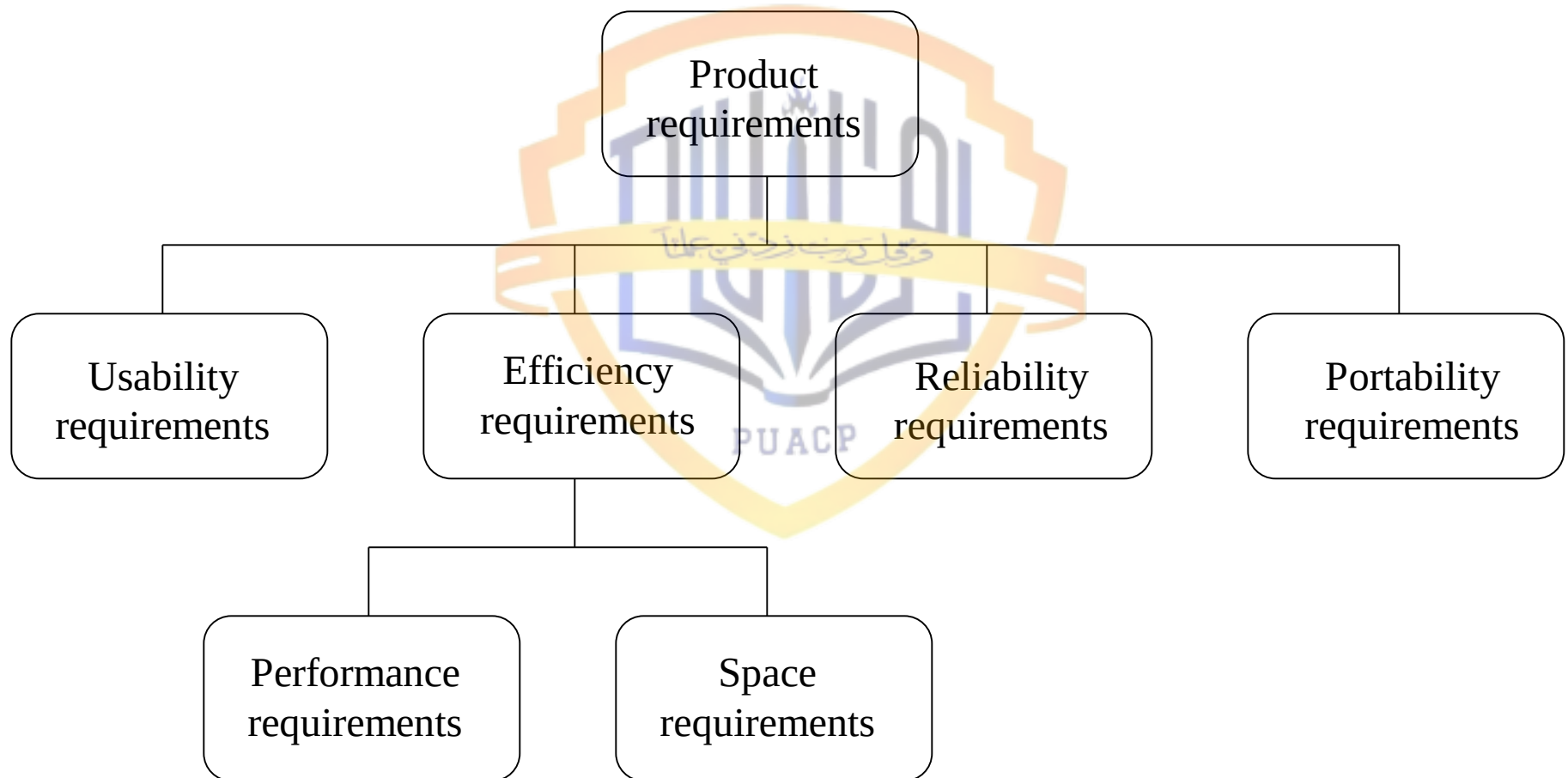
Non-Functional Requirements - 3

- For example, if an aircraft system does not meet reliability requirements, it will not be certified as 'safe'
- If a real-time control system fails to meet its performance requirements, the control functions will not operate correctly

Non-Functional Requirements - 4



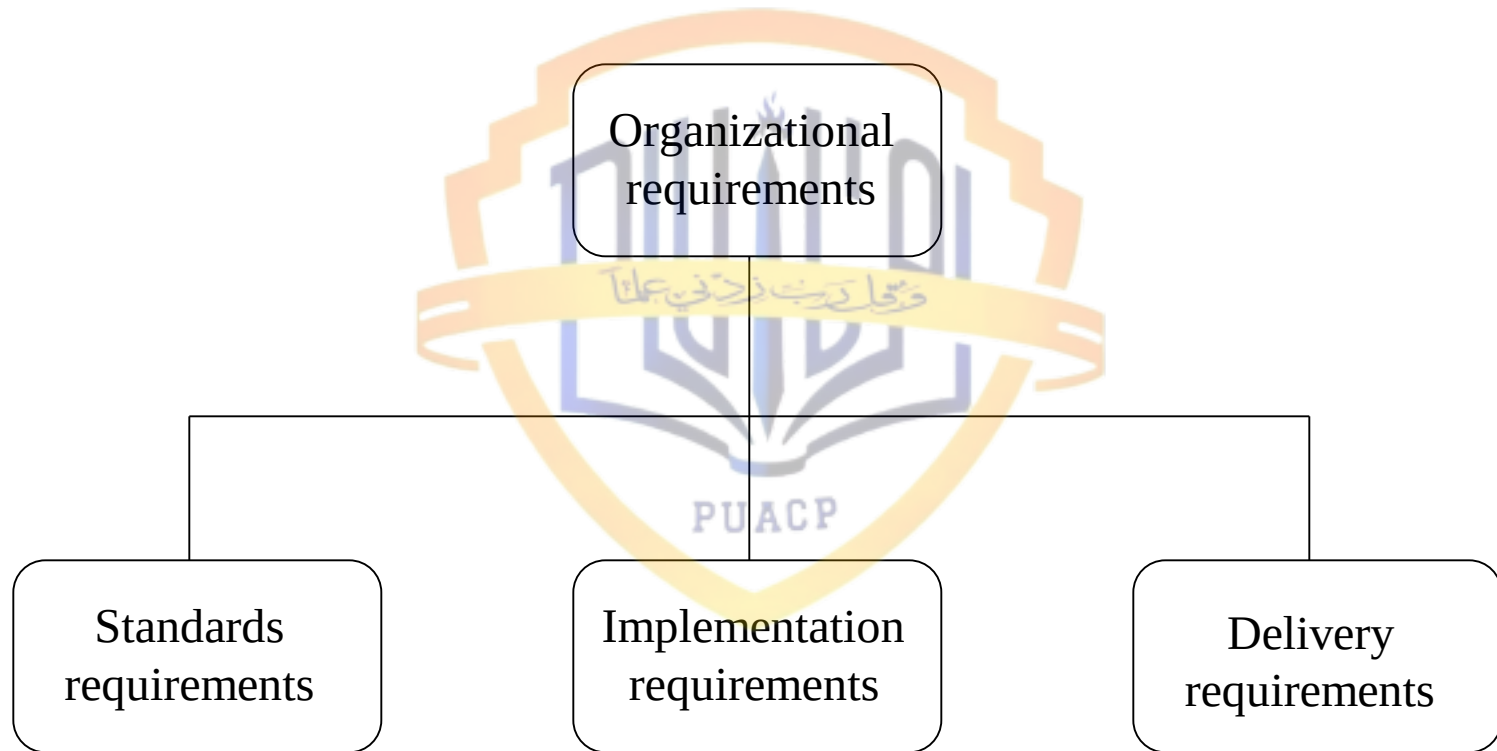
Product Requirements - 1



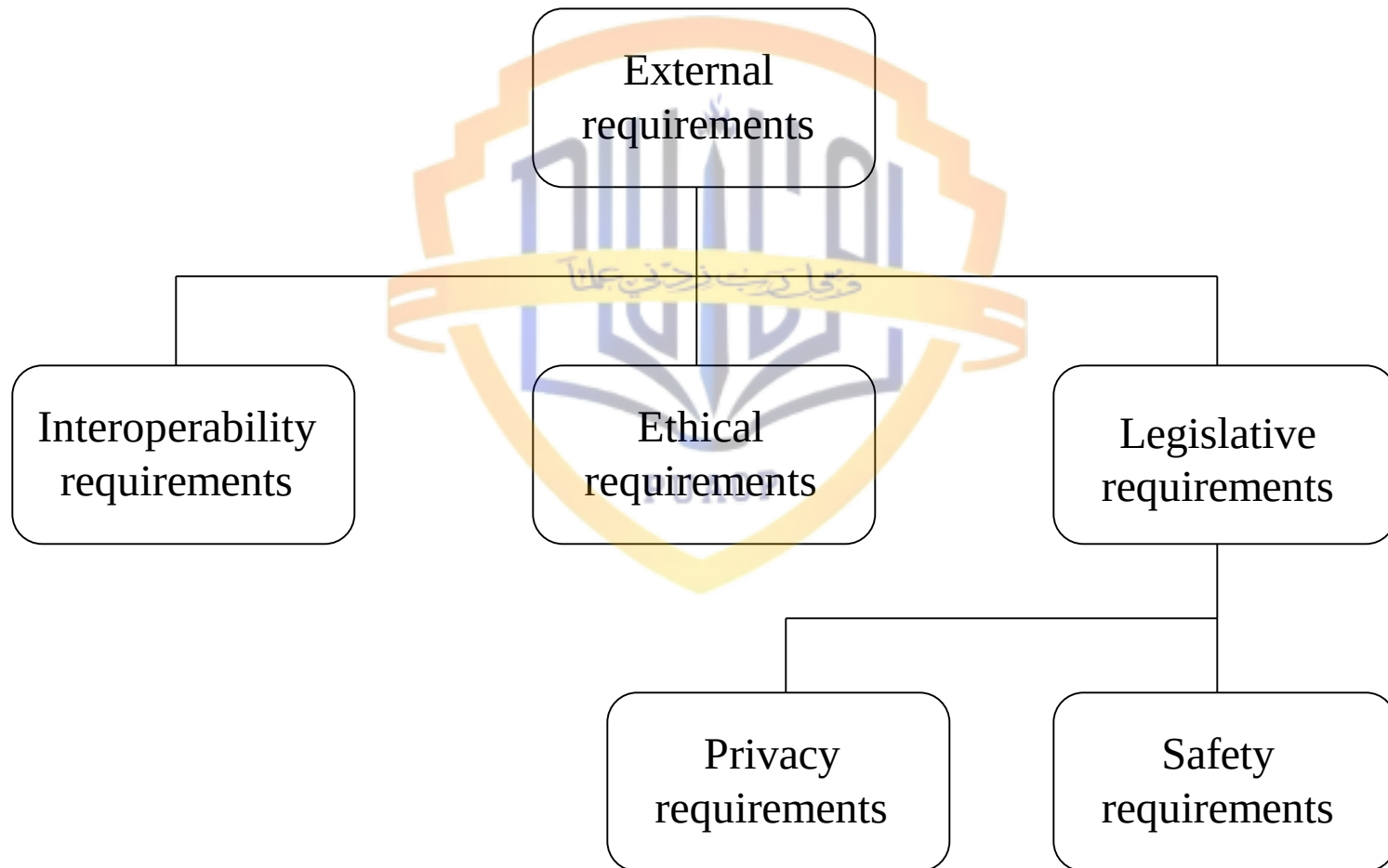
Product Requirements - 2

- The system shall allow one hundred thousand hits per minute on the website
- The system shall not have down time of more than one second for continuous execution of one thousand hours

Organizational Requirements - 1



External Requirements - 1



External Requirements - 2

- The system shall not disclose any personal information about members of the library system to other members except system administrators
- The system shall comply with the local and national laws regarding the use of software tools

Observations on Non-Functional Requirements - 1

- Non-functional requirements are written to reflect general goals for the system.

Examples include:

- ☐ **Ease of use**
- ☐ **Recovery from failure**
- ☐ **Rapid user response**

Observations on Non-Functional Requirements - 2

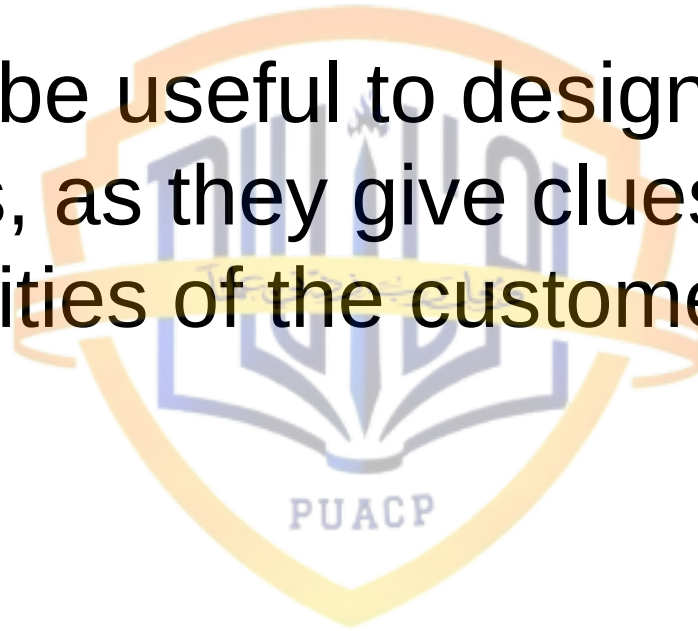
- Goals are open to misinterpretation
- Objective verification is difficult
- Distinction between functional and non-functional is not always very clear

Observations on Non-Functional Requirements - 3

- Non-functional requirements should be written in a quantitative manner as much as possible, which is not always easy for customers
- For some goals, there are no quantitative measures, e.g., maintainability

Observations on Non-Functional Requirements - 4

- Goals can be useful to designers and developers, as they give clues to them about priorities of the customers

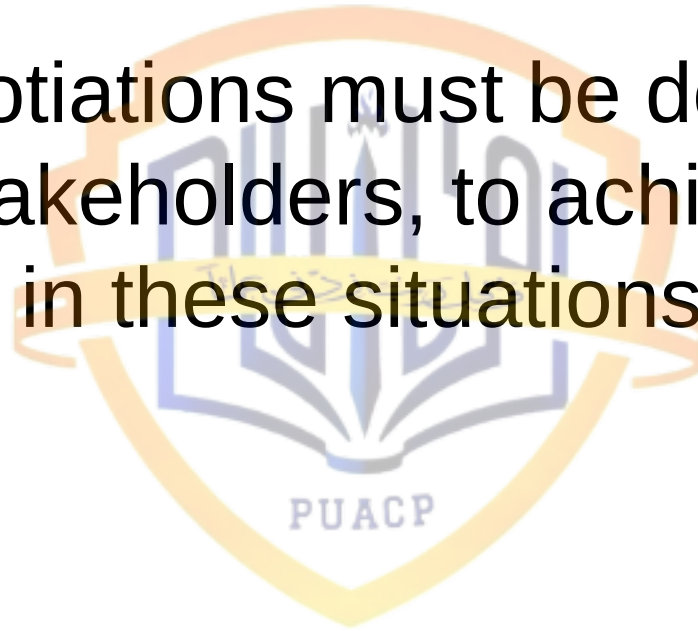


Observations on Non-Functional Requirements - 5

- Chances of conflicts within non-functional requirements are fairly high, because information is coming from different stakeholders. For example, different stakeholders can give different response times or failure tolerance levels, etc.

Observations on Non-Functional Requirements - 6

- Some negotiations must be done among different stakeholders, to achieve an agreement in these situations



Observations on Non-Functional Requirements - 7

- Non-functional requirements should be highlighted in the requirements document, so that they can be used to build the architecture of the software product

Domain Requirements - 1

- Requirements that come from the application domain and reflect fundamental characteristics of that application domain
- Can be functional or non-functional
- These requirements, sometimes, are not explicitly mentioned, as domain experts find it difficult to convey them. However, their absence can cause significant dissatisfaction

Domain Requirements - 2

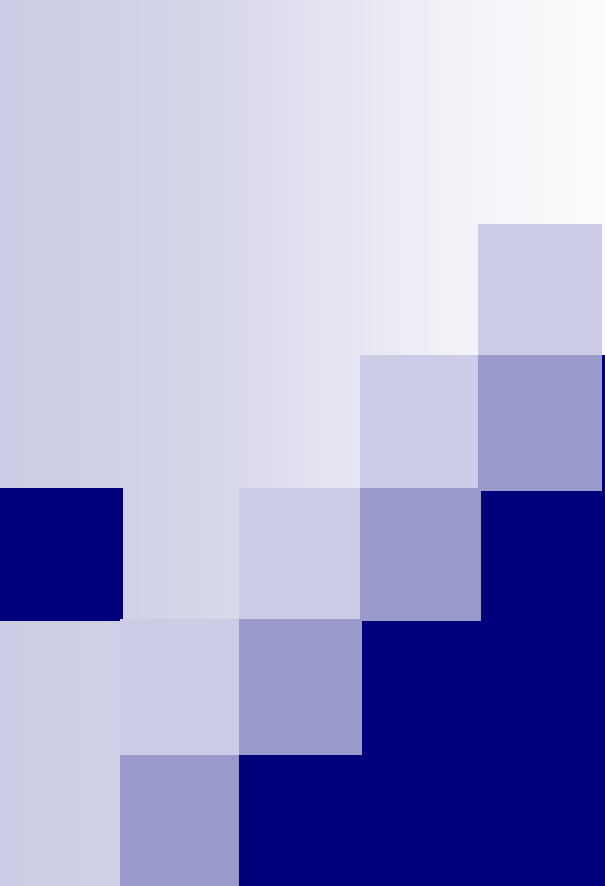
- Domain requirements can impose strict constraints on solutions. This is particularly true for scientific and engineering domains
- Domain-specific terminology can also cause confusion

Inverse Requirements

- They explain what the system shall **not** do. Many people find it convenient to describe their needs in this manner
- These requirements indicate the indecisive nature of customers about certain aspects of a new software product

Design and Implementation Constraints - 1

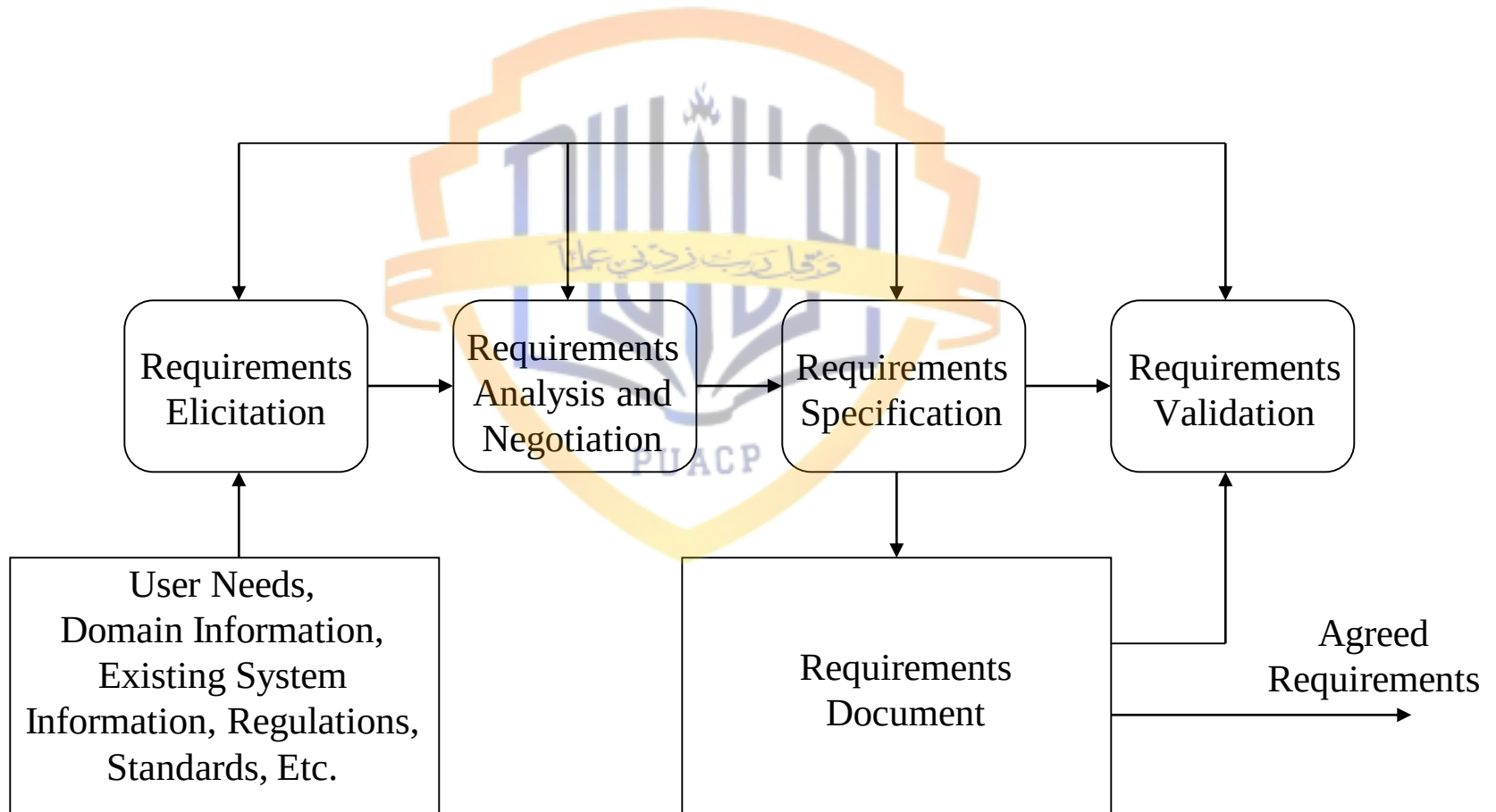
- They are development guidelines within which the designer must work, which can seriously limit design and implementation options
- Can also have impact on human resources



Requirements Engineering Process



Requirements Engineering Activities



Requirements Elicitation

- Determining the system requirements through consultation with stakeholders, from system documents, domain knowledge, and market studies
- Requirements acquisition or requirements discovery

Requirements Analysis and Negotiation - 1

- Understanding the relationships among various customer requirements and shaping those relationships to achieve a successful result
- Negotiations among different stakeholders and requirements engineers

Requirements Analysis and Negotiation - 2

- Incomplete and inconsistent information needs to be tackled here
- Some analysis and negotiation needs to be done on account of budgetary constraints

Requirements Specification

- Building a tangible model of requirements using natural language and diagrams
- Building a representation of requirements that can be assessed for correctness, completeness, and consistency

Requirements Document

- Detailed descriptions of the required software system in form of requirements is captured in the requirements document
- Software designers, developers and testers are the primary users of the document

Requirements Validation - 1

- Reviewing the requirements model for consistency and completeness
- This process is intended to detect problems in the requirements document, before they are used as a basis for the system development

Requirements Validation - 2

- Requirements need to be validated for
 - consistency
 - testability
 - performance issues
 - conformance to local/national laws
 - conformance to ethical issues
 - conformance to company policies
 - availability of technology

Requirements Management

- Identify, control and track requirements and the changes that will be made to them
- It is important to trace requirements both ways
 - origin of a requirement
 - how is it implemented
- This is a continuous process

References

- Software Quality: Analysis and Guidelines for Success by Capers Jones
- Requirements Analysis and Specification by Alan M. Davis
- A Practitioner's Approach to Software Engineering by Roger Pressman
- Software Engineering 6th Edition, by I. Sommerville, 2000
- 'Software Engineering Quality Practices' by R. K. Kandt, Auerbach Publications, 2006